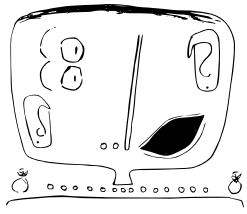


CUDA-Mode Session 4: Compute and Memory basics (based on ch 4 + 5 of the PMPP book)

3 February 2024

Thomas Viehmann, tv@lernapparat.de, MathInf GmbH

And starting next week: ...

CPU:

```
for(int i=0; i<320; i++)
    C[i] = A[i] + B[i]
```

GPU: create 320 threads

thread[i]: C[i] = A[i] + B[i]

(i=0→319)

then GPU organizes those threads into groups of 32

warp i (0→9): threads 0–31 ⋮ threads 288→319

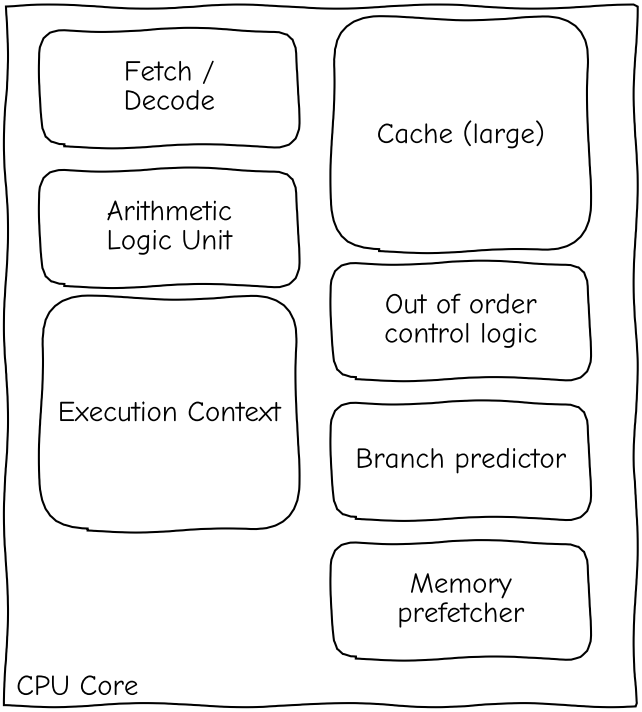
Chapter 4: Compute Architecture and Scheduling
aka: How to keep all of the GPU busy

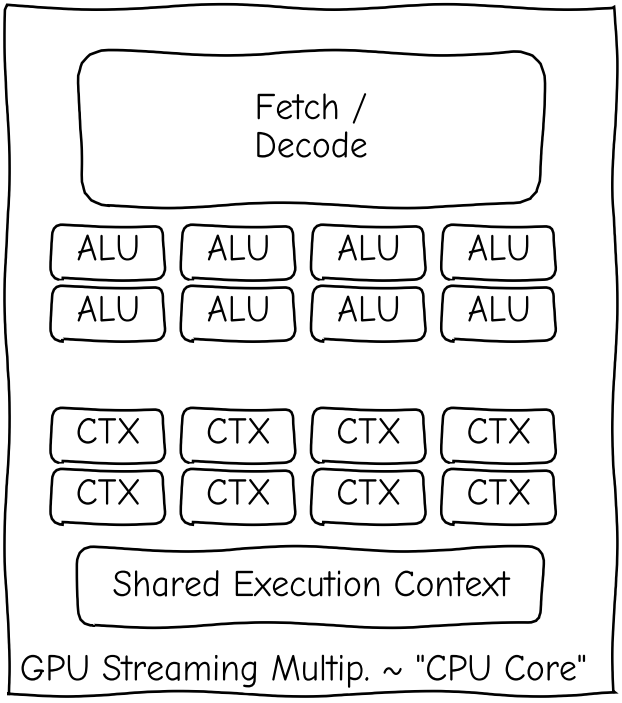
on the hardware's side:

Kernel launch
- Grid
  - Thread blocks
    - Warps
      - 32 threads

GPU
- Streaming Multiprocessors (SM)
  - Warp Schedulers
    + execution units
    + registers
    + shared memory

CPU core ~ GPU Streaming Multiprocessor (comic version)





Actually 1SM ~ 4 “CPU-Cores”,
originally (<=Pascal) GPU threads in a warp shared a Program Counter, now each has one

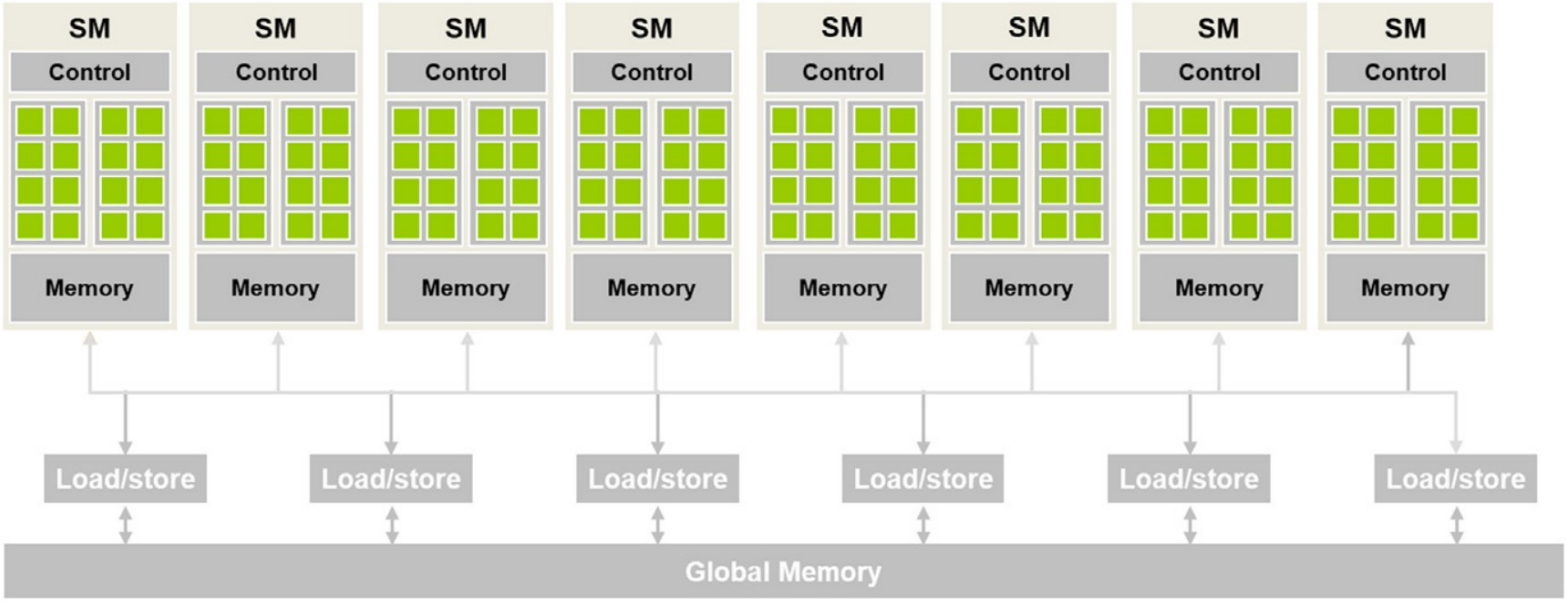


FIGURE 4.1

Architecture of a CUDA-capable GPU.

Inside the GPU
(here RTX3090)

82 SMs
(=Streaming
Multiprocessor)

1 common
L2 cache

Almost no FP64
in consumer/
non-datacenter
GPUs
(2 FP64 per SM vs.
128 FP32)

The foundational mental model

Think of an SM as a restaurant kitchen:

- Threads are individual food orders.
- A warp is a tray containing 32 similar orders.
- Blocks are groups of trays assigned to one kitchen.

tures 168 FP64 units (two per SM), which are not depicted in this
is 1/64th the TFLOP rate of FP32 operations. The small number of FP64
nsure any programs with FP64 code operate correctly, including FP64

- The warp scheduler chooses a tray that is ready to be worked on.
- Registers and shared memory are the kitchen's limited counter space.

Streaming Multiprocessor

"SM is a worker that execute warps"

- A thread block is assigned to one SM
  (max 1536 threads assignable to SM)
  NO control which block in a grid when where
  (ok, in Hopper+ we can have thread block groups)
  ↳ 64 threads per block
  thread / 64 = block
- 4 warps or"part-warp" can compute at a cycle, those SHARE one instruction (but Volta+ does have per-thread program counter)
- 32 FP32 units (one per thread), 16 of which know INT32 → thread / 32 = warps
- 16k 32-bit registers shared between things scheduled on the same block)
- L1 cache and shared memory share hardware (128KB) directly on the SM
  shmem can be 0/8/16/32/64/100KB
  L1 Cache the remainder (>=28KB)

multiple blocks simultaneously if it

Figure 3. GA10x Streaming Multiprocessor (SM)

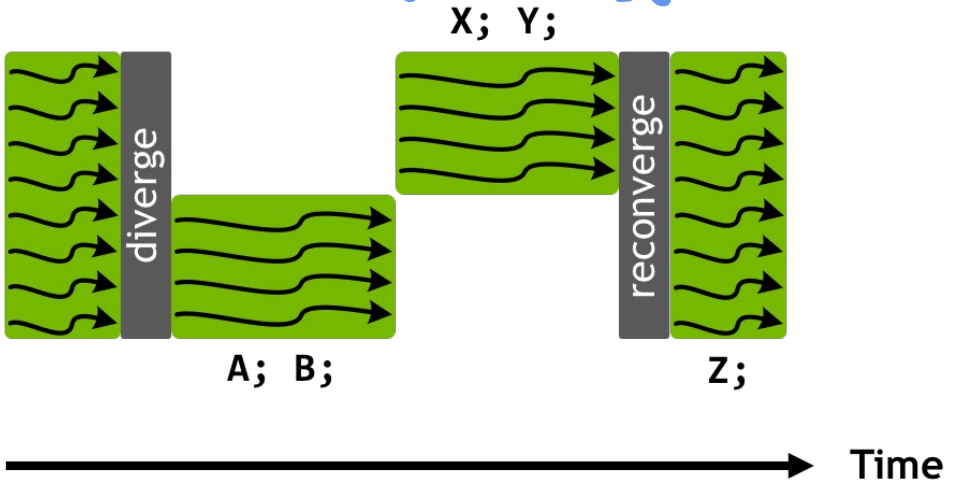
thread capacity

"each thread calculates it global array pos."

Warp Divergence (<= Pascal)

```
int i = blockIdx.x * blockDim.x
      + threadIdx.x
C[i] = A[i] + B[i]
```

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



Old way: threads share program counter, but have an "active mask".

Careful not to do inter-thread communication / sync inside if (without mask)!

Automatic reconvergence.

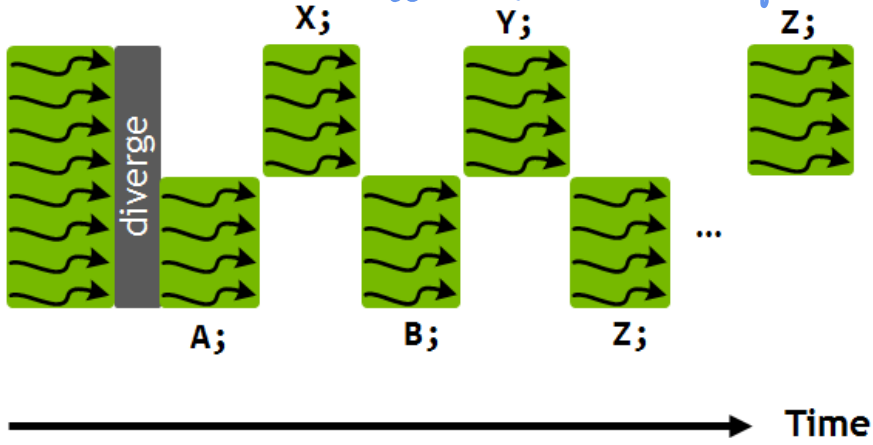
N.B.: As there are load/store instructions the typical pattern (cond ? x[i]: 0f) will not cause divergence.

Warp Divergence occur when threads within the same warp follow different control flow paths

Warp Divergence (>= Volta)

diff. warps allowed to follow diff. paths

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



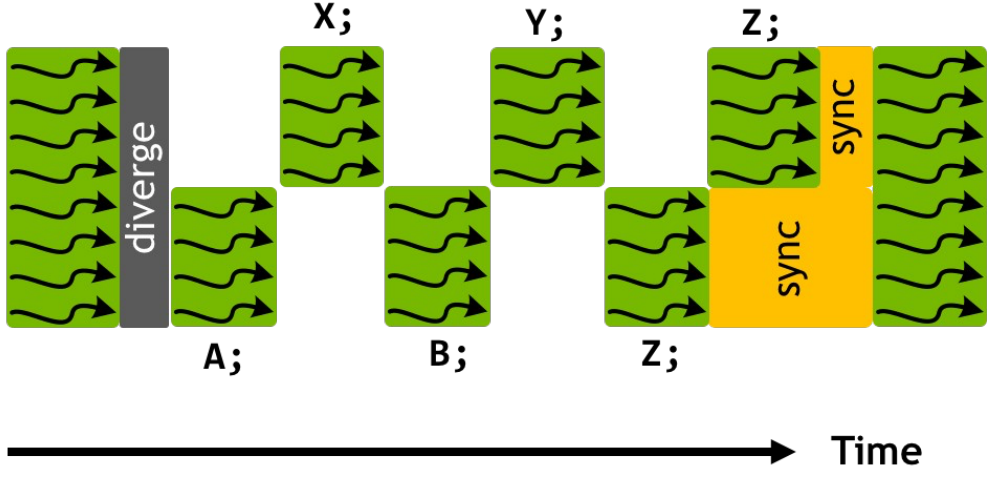
New way: threads have individual program counter, GPU will threads of a warp by PC for execution.
No automatic reconvergence, but potentially better latency hiding (e.g. if both parts load from dram).

"Branches become inefficient when they divide threads of a single warp"

=> "warp is most efficient when all 32 threads performs approx. the same cost"

Warp Divergence (>= Volta)

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
__syncwarp()
```



No automatic reconvergence → use __syncwarp to rejoin warp.
Inter-Thread communication (e.g. shuffle) will also sync the participating threads.
__syncthreads() syncs an entire block, not just warps.

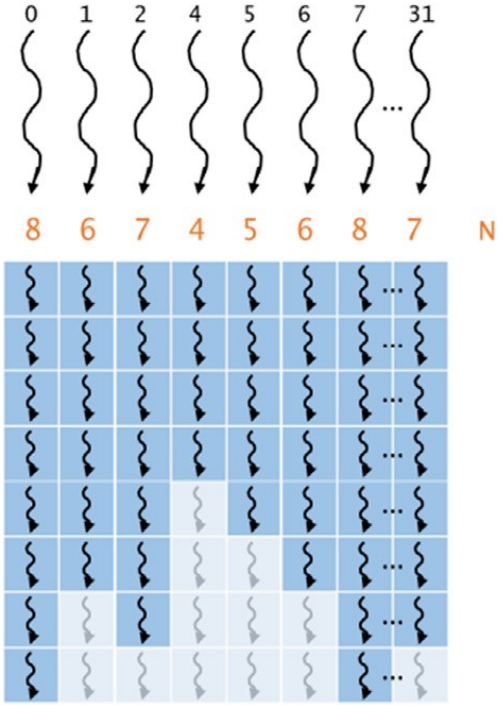
We're not done with a warp until everything is done

(note: this is a pre-volta image)

```
N = a[threadIdx.x];
for(i = 0; i < N; ++i) {

    A

}
```



Loops with variable upper bound also cause divergence.

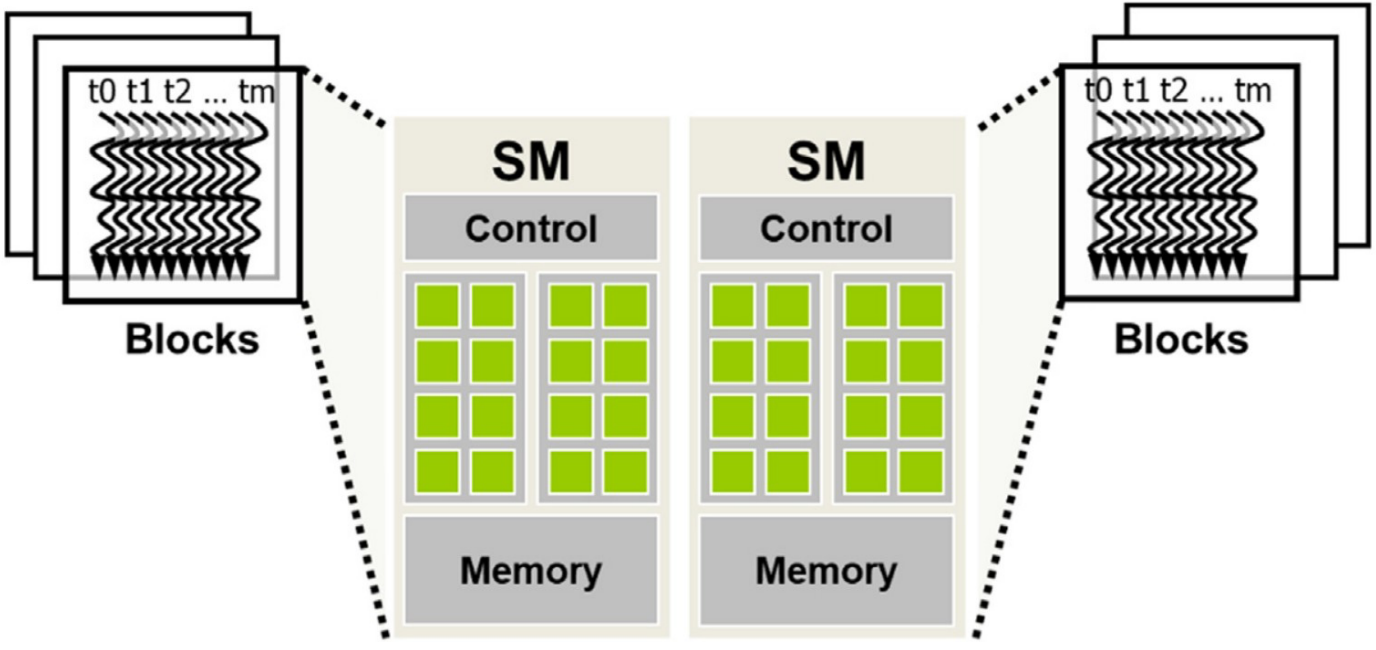


FIGURE 4.2

Thread block assignment to streaming multiprocessors (SMs).

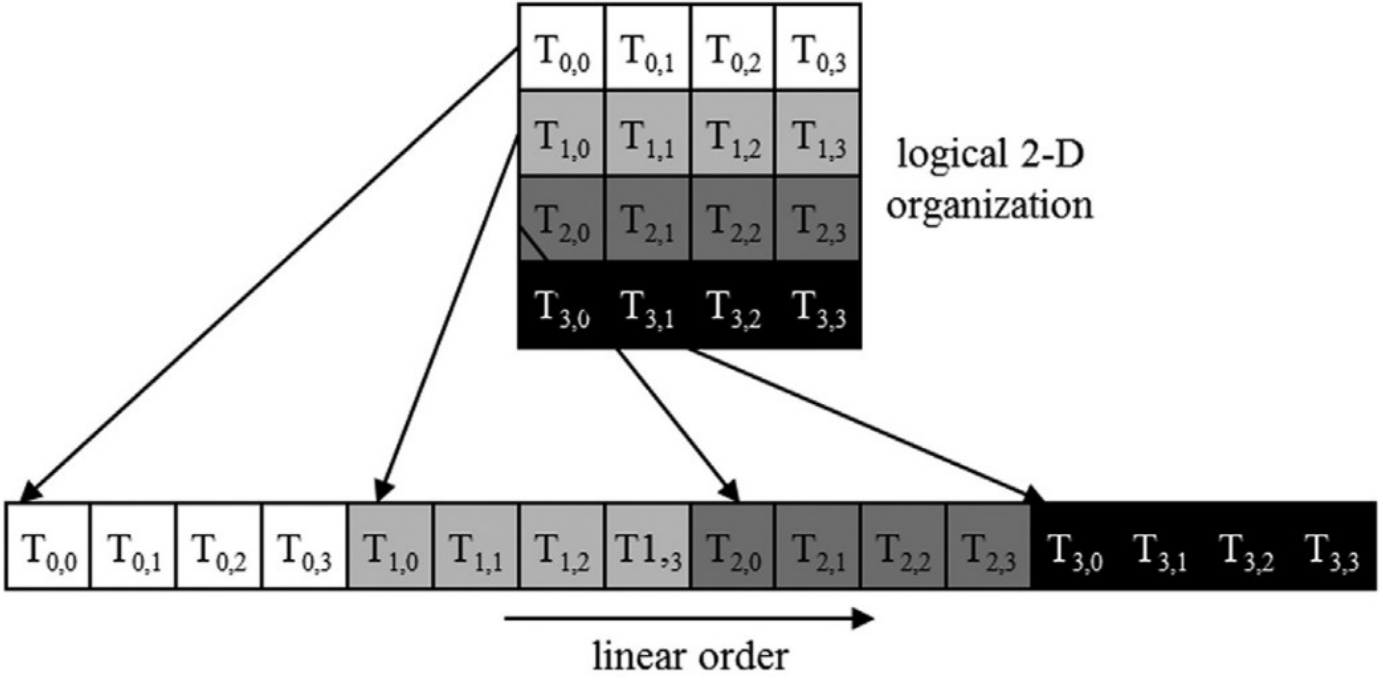


FIGURE 4.7

Placing 2D threads into a linear layout.

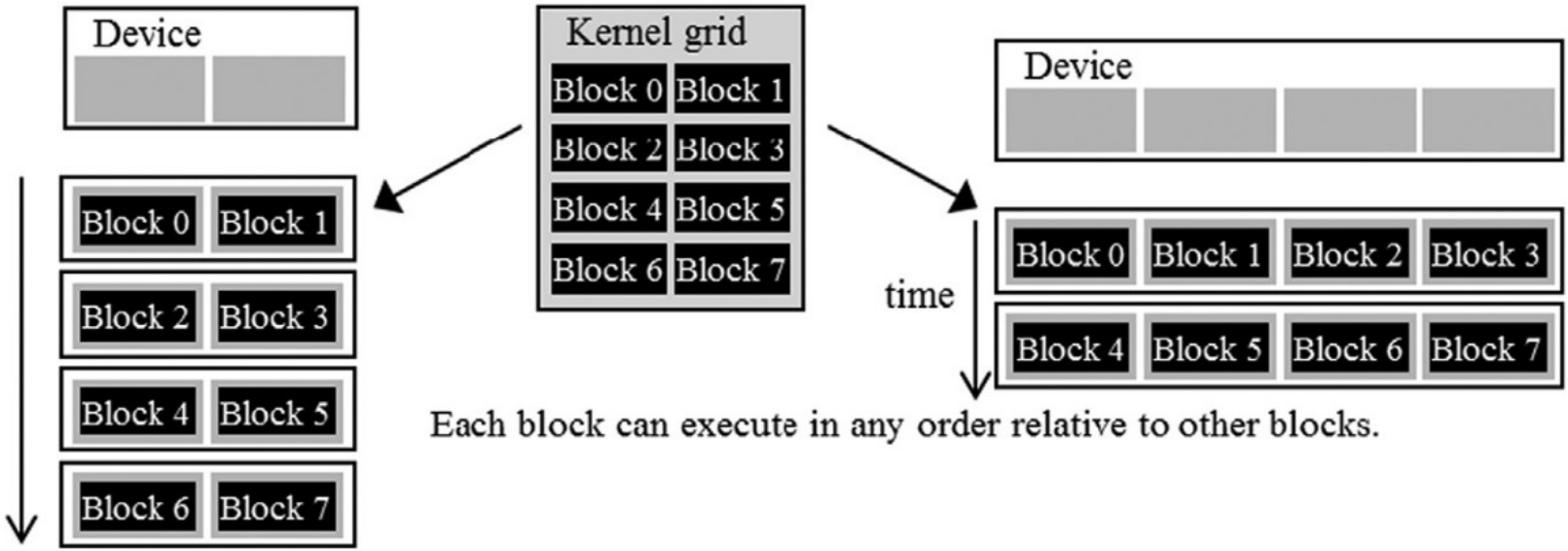


FIGURE 4.5

Lack of synchronization constraints between blocks enables transparent scalability for CUDA programs.

$\text{occupancy} = \frac{\text{resident warps on the SM}}{\text{max. supported resident warps}}$

Getting good occupancy – balance resources

Ex: - SM supported 64 (max) res. warps
- each block contain 256 th. = 8 warps
- resource usage allows 4 blocks to reside simultaneously

$4 \times 8 = 32$

$\frac{32}{64} \times 100 = 50\%$ occupancy

- Have 82 SM → many blocks = good
  (for comparison Jetson Xavier has 8 Volta SM)
- Can schedule up to 1536 threads per SM
  → power of two block size <512 desirable
  (some other GPUs 2048)
- Avoid divergence to execute an entire warp (32 threads) at each cycle
- Avoid FP64/INT64 if you can on Gx102 (GeForce / Workstation GPUs)
- Shared Memory and Register File → limits number of scheduled on SM
  (use __launch_bounds__ / C10_LAUNCH_BOUNDS to advise compiler of # of threads for register allocation, but register spill makes things slow)

"Occupancy is not the percentage of GPU business"

- Previously, there was an excel sheet for occupancy calc, now it is in the Nsight Compute

```
text

Thread → one piece of work
32 threads → one warp
```

>) to get properties (e.g. max_threads_per_multi_processor)

e/DevZone/docs/html/C/doc/html/group__CUDART__DEVICE_g5

```
Warps form blocks
One block stays on one SM
An SM schedules ready warps
Other warps hide waiting latency
Resources determine occupancy
```

Chapter 5: Memory architecture and data locality

aka the basics of getting fast kernels

How do PyTorch programs spend their time

At a very high level:

- Python processing,
- Data "administrative overhead" (allocate Tensor structures etc.),
- Data acquisition (I/O), ← check this before diving into GPU optimization
- The GPU computation
  - Fixed cost (kernel launches etc.)
  - Memory access (read inputs / write results), ← This is us, chapter 5, i.e. now.
  - "real" computation ("FLOPs"). ← occupancy a key part, chapter 4

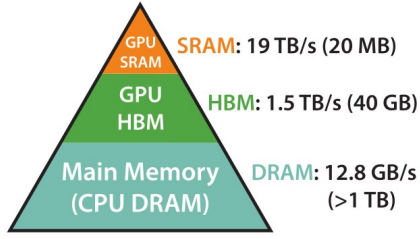
Thomas Rules of Thumb

- As long as you don't have close to 100% GPU utilization in nvidia-smi, work on data acquisition etc.
- As long as you have Tensors with a few 100s of elements, "Python is slow" and data administrative over head is single digit percentages.
- Obviously the algorithms also matter (parallel algorithms in the following chapters)

But we are beyond this here!

Memory access as a bottleneck

- Eager PyTorch does “load input, compute, store output” for each operation.
- Obviously faster if we can combine kernels:
  “load inputs, compute, compute, compute, compute, compute, compute, store outputs”
- Has been in the focus of PyTorch for a long time
  - original reason for the PyTorch JIT – fusing pointwise operations into one kernel has been key e.g. to get LSTMs close to CuDNN perf.
  - 2nd gen PyTorch JIT fusers added contractions etc. (NVFuser going beyond PyTorch in https://github.com/NVIDIA/Fuser and learning new things every week)
  - Today’s inductor / Triton based optimizations are also partly with that, but supports more complex ops
- Also core ingredient of flash attention (along with other things)
  (graphic on the right from Dao et al., Flash Attention…) →



Memory Hierarchy with Bandwidth & Memory Size

Memory and computation

memory hierarchy

fastest smallest
- Registers → private to one thread
- shared memory / L1 cache → located near SM
- L2 cache → shared across the GPU

slowest largest
- Global GPU memory → large but relatively slow

- Take rgb2gray: For each pixel
  - Load 3 bytes,
  - Compute I (1 mul + 1 add in 32 bit integer)
  - compute 5 ops (3 mul, 2 add – ideally in 32 bit) + data conversion,
  - Store 1 byte

What speed can we expect maximally for a 2048 x 2048 image?
RTX3090:

- NVidia lists ~900GB/s memory bandwidth 4*4M bytes transfer: ~18µs (16M/900GB) s ("speed of light")
- Compute: 35.6FP32 TFLOP/s 16.8 Int32 TFLOP/s ~2µs (generous).
  (Does not include latency hiding across warps.)
- If measuring with %timeit: Kernel Launch: ~3µs (see with empty kernel)
- (don't forget the element size if using 32bit or 16bit)

Measured time of kernel (with "f" for the consts): 27µs ~ 74% of theoretically possible.
Note: I have made an "out" function to split out the memory allocation, but it is relatively fast as long as you have the caching allocator.
N.B. Alignment helps... (try copy kernel with offset)

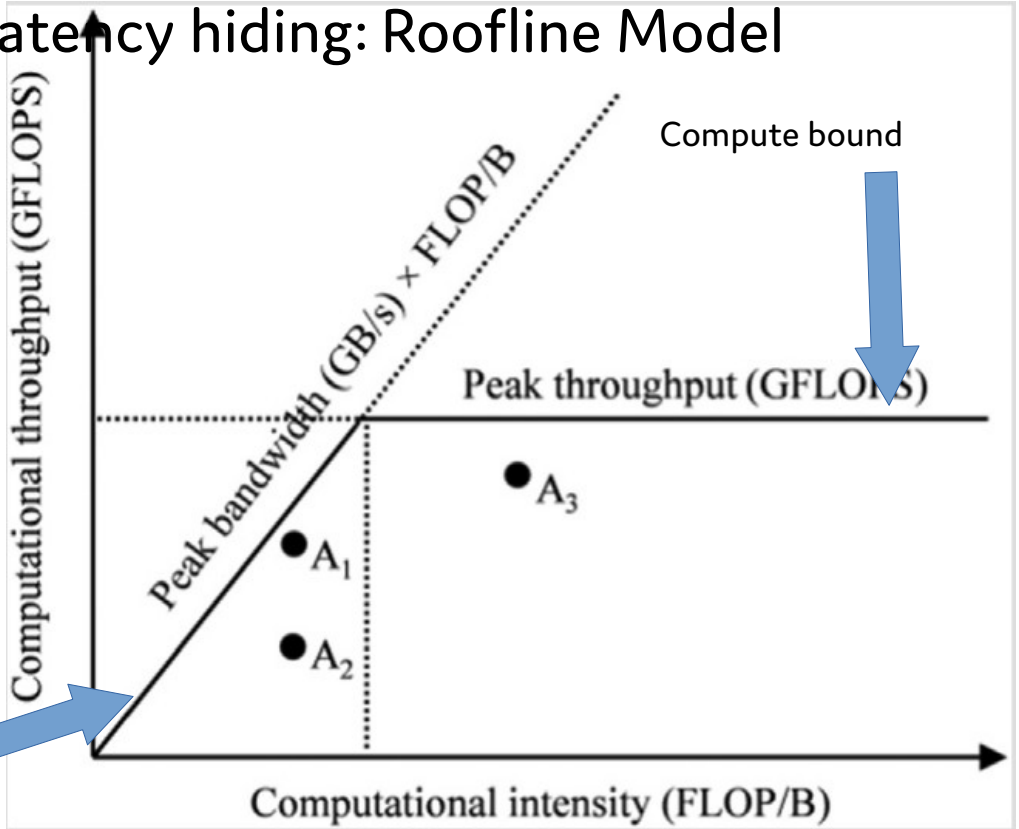
memory bound: performance is limited by the ability to access data from memory
compute bound: computer's performance is limited

by its computational capacity

Similar model with latency hiding: Roofline Model

Key quantity: **computational intensity**: FLOP/Byte of memory transfer

Latency hiding: having multiple warps on the SM allows warps to compute while others wait
(i.e. the memory transfer "+" compute becomes a max(..., ...))
→ roofline model



Memory bound

Device code can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

Host code can

- Transfer data to/from per grid **global** and **constant memories**

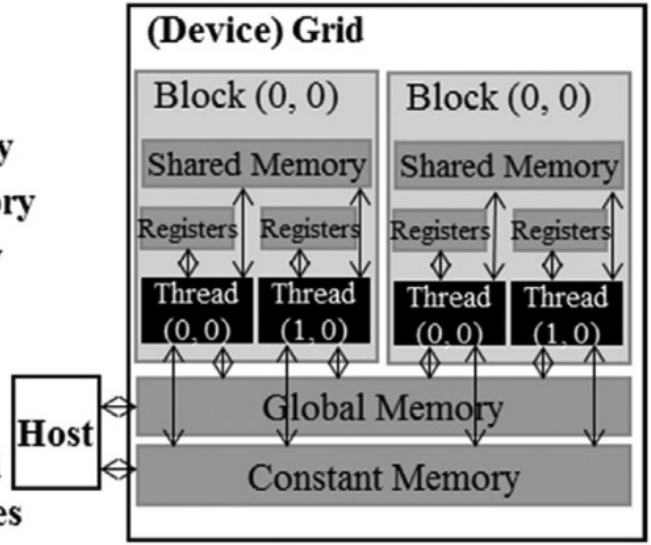


FIGURE 5.2

An (incomplete) overview of the CUDA device memory model. An important type of CUDA memory that is not shown in this figure is the texture memory, since its use is not covered in this textbook.

→ A register belongs to one thread, so other threads cannot read it directly

→ Shared memory belongs to the block, so it's useful when several threads needs the same

Table 5.1 CUDA variable declaration type qualifiers and the properties of each type.

→ global memory is more expensive than reg or shmem for reuse

Variable declaration	Memory	Scope	Lifetime
Automatic variables other than arrays	Register	Thread	Grid
Automatic array variables	Local	Thread	Grid
__device__ __shared__ int SharedVar;	Shared	Block	Grid
__device__ int GlobalVar;	Global	Grid	Application
__device__ __constant__ int ConstVar;	Constant	Grid	Application

What kinds of memory does the rgb2gray kernel use?

Tiling: breaks the matrices into small

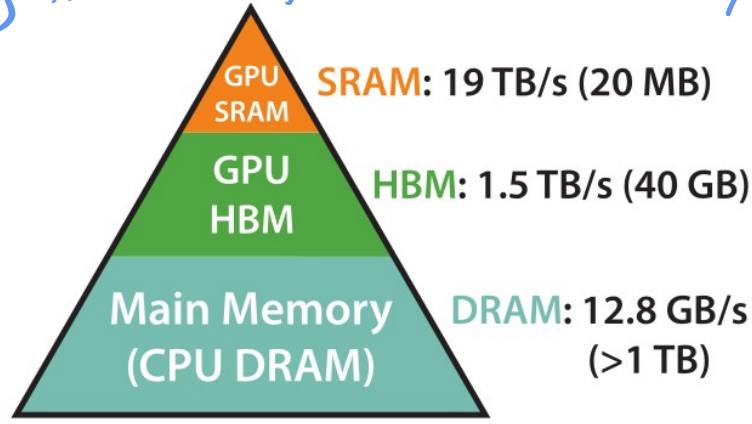
sections that fit in shared memory.

Tiling – why?

"reduces repeated global-memory accesses by increasing the reuse of data stored in shared memory"

- In Matmul: each of the n^2 outputs uses 2n inputs
- every input used n times, naively n times from main memory
  → inefficient
  → try to reuse param

Similar in Convolution, FlashAttention,...



Memory Hierarchy with Bandwidth & Memory Size

Dao et al. Flash Attention...

Tiling

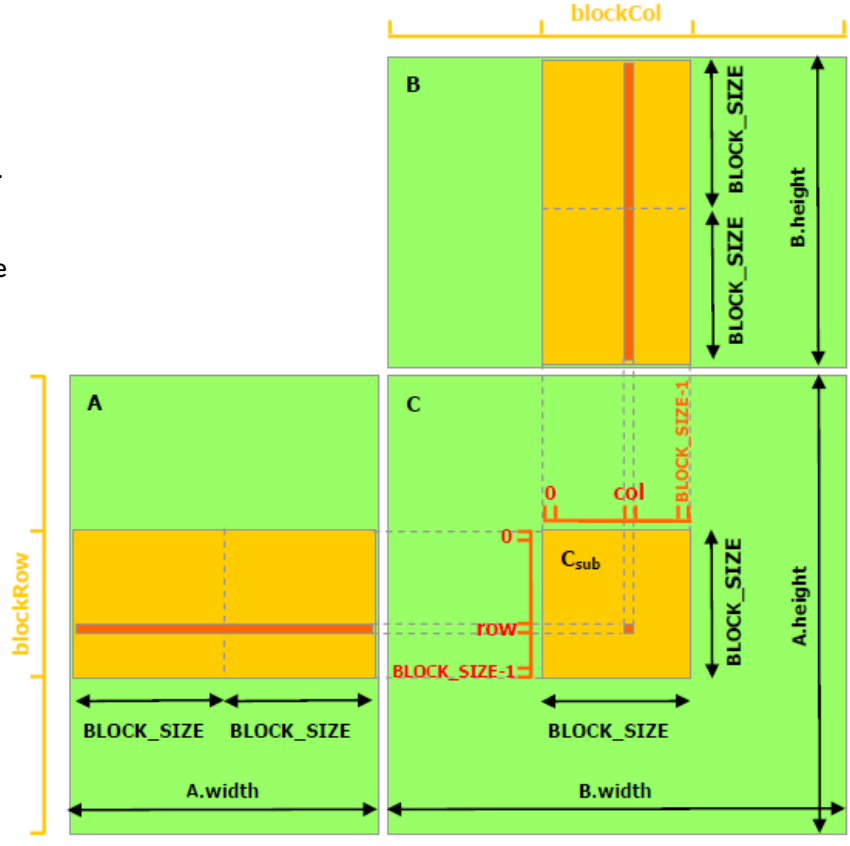
Divide output and input matrix into “tiles” (e.g. 16x16).

Assuming TILE_SIZE divides n:
Output tile depends on 2n/TILE_SIZE input tiles of size TILE_SIZE*TILE_SIZE.

Have $(n/TILES\_SIZE)^2$ tiles:
read each input only n/TILE_SIZE times from main memory.

...need to store input tiles in shmem, so they can be used in TILE_SIZE computations by various threads in the block.

Easiest setup: $TILE\_SIZE^2$ threads



Practical implementation of tiling

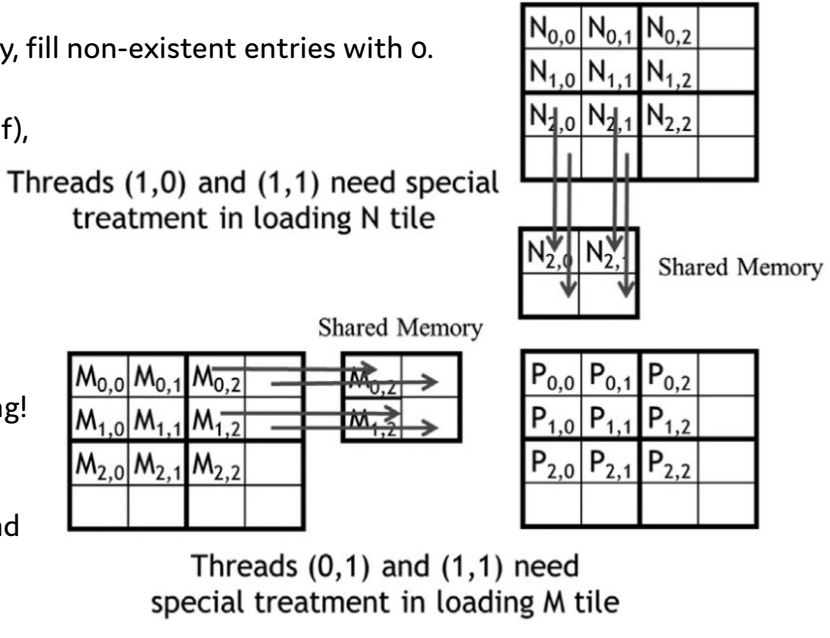
Tile padding if TILE_SIZE does not divide n

- Do this when reading / writing global memory, fill non-existent entries with 0.
- I like to write this as a tenary (valid ? x[i] : 0.0f), makes me feel good w.r.t. divergence (will be translated into conditional load also if you use if)

Balance shmem use for occupancy!

Don't forget **__synthreads before and after** reading! (Q: why?)

Will later consider how much work to do per thread (thread coarsening).

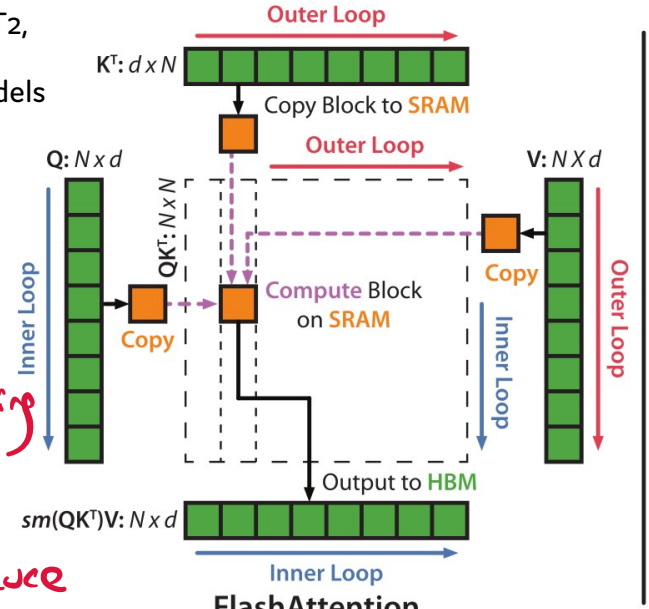


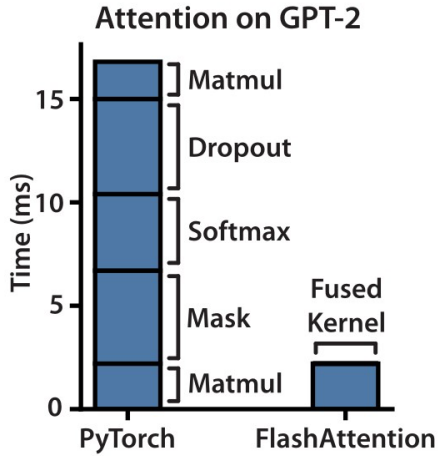
Advanced tiling and fusing: FlashAttention

3x end-to-end speedup on GPT2,

high impact also for larger models

Exercise: Implement
flash attention from scratch.

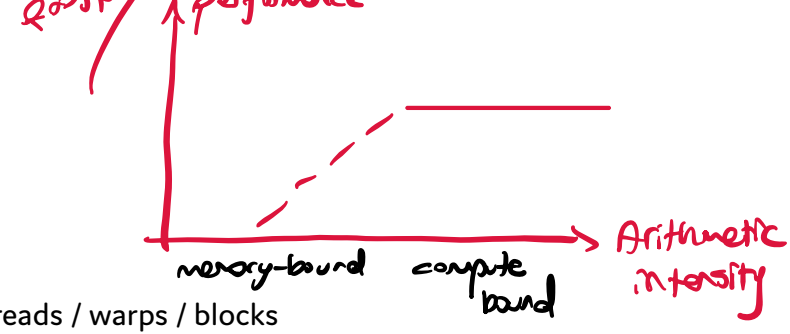




Occupancy and scheduling help hide latency

Tiling and reuse reduce bandwith demand

Conclusion



Key takeaways from ch 4 and 5:

- how the GPU organizes the computation in threads / warps / blocks
- use as much of the hardware as possible (occupancy), balance bottlenecks
- avoid thread divergence
- roofline model and "theoretical maximum speed"
- try to not read/write too much from/too global memory.

Next chapter: read/write consecutive and aligned global memory locations (**coalesced memory access**)
(and a lot about how this works in detail)

Sources

- The book (Wen-mei W. Hwu, David B. Kirk, Izzat El Hajj: Programming Massively Parallel Processors)
- GA102 Whitepaper (but check properties against query) https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf
- Inside Volta for the independent thread scheduling https://developer.nvidia.com/blog/inside-volta/
- Tri Dao et al.: FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, https://arxiv.org/abs/2205.14135

Page 7

Threads, Warps, Blocks

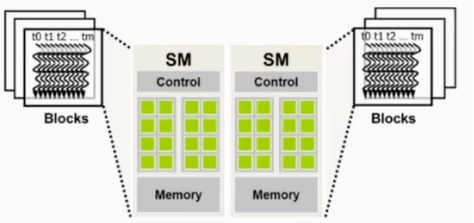


FIGURE 4.2
Thread block assignment to streaming multiprocessors (SMs).

Launching a CUDA kernel with
- block layout (=how many threads in a block)
- grid layout (= how many blocks should be launched).

Threads in a block: executed in parallel on the same SM (and can access its shared memory).
Blocks are (exceptions on very new GPUs) completely **independent**:
- CUDA is free to arbitrarily assign blocks to Sms,
- we don't know about execution order.

Thread block run on an SM is divided into **Warps** of 32 threads
- each warp run on a (fixed) of the SMs processing units,
- all warps simultaneously assigned to that processing unit take turns, but registers stay.

Aside: on AMD hardware and terminology, Warps are Wavefronts of size 64 (by default(?))

Page 8

Thread linearization and division into warps

$T_{0,0,0}$	$T_{1,0,0}$	$T_{2,0,0}$	…	$T_{bd.x-1,0,0}$
$T_{0,1,0}$	$T_{1,1,0}$	$T_{2,1,0}$	…	$T_{bd.x-1,1,0}$
$T_{0,2,0}$	$T_{1,2,0}$	$T_{2,2,0}$	…	$T_{bd.x-1,2,0}$
…	…	…		…
$T_{0,bd.y-1,0}$	$T_{1,bd.y-1,0}$	$T_{2,bd.y-1,0}$		$T_{bd.x-1,bd.y-1,0}$

Logical 2d/3d organization

Linear order (thead id in Block= threadIdx.x + blockDim.x * threadIdx.y + blockDim.y * blockdim.x * threadIdx.z)

$T_{0,0,0}$ $T_{1,0,0}$ $T_{2,0,0}$ … $T_{bd.x-1,0,0}$ $T_{0,1,0}$ $T_{1,1,0}$ $T_{2,1,0}$ … $T_{bd.x-1,1,0}$

warp of 32 threads	warp of 32 threads	warp of 32 threads

The linearly ordered threads are grouped into warps of 32 threads each

We write the indices as $T_{threadIdx.x,\ threadIdx.y,\ threadIdx.z}$ bd is the shorthand for blockDim
(Note that this differs slightly from the Book's figure 4.7.)